



Home

Exerciții propuse:

Laborator: Sintaxă abstractă

- Back to [PLP - Fall 2024](#)

Exerciții de antrenament:

- Lucrul cu șiruri de caractere în Coq:

```
Require Import String.  
(* Check "a". throws 'Error: No interpretation for string "a".' *)
```

Pentru a utiliza șiruri de caractere:

```
Open Scope string_scope.  
Check "a".  
Check "".  
Check "abcb".
```

Exerciții propuse:

1. (1 punct) Definiți sintaxa expresii aritmetice care includ: (1) numere naturale, (2) variabile, (3) adunări, (4) scăderi, (5) împărțiri. Pentru acestea veți crea un tip de date inductiv AE_{Exp} care va avea constructorii: `num`, `var`, `add`, `sub`, `div` (enumerati în ordinea corespunzătoare operațiilor 1,2,3,4,5). Utilizați Coercion și Notation astfel încât arborii expresiilor de mai jos să corespundă convențiilor matematice cunoscute (e.g., priorităților dintre +, - și /, asociativitate la stânga pentru operatori binari):

```
Check (add (num 2) (add (var "x") (div (num 2) (var "y")))).  
Check (add 2 (add "x" (div 2 "y"))).
```

```
Set Printing Coercions.  
Check (add 2 (add "x" (div 2 "y"))).  
Unset Printing Coercions.
```

```
Check 2 + ' ("x" + ' 2 /' "y").  
Set Printing All.  
Check 2 - ' ("x" + ' 2 /' "y").  
Check 2 - ' "x" /' "y".  
Check "x" /' "y" + ' 2.  
Check "x" /' "y" /' "z".  
Unset Printing All.
```

2. (1 punct) Se consideră următoarea definiție inductivă pentru condiții booleene:

```
Inductive Cond :=
| base : bool -> Cond
| bnot : Cond -> Cond
| beq  : AExp -> AExp -> Cond
| less : AExp -> AExp -> Cond
| band : Cond -> Cond -> Cond.
```

Utilizați Coercion și Notation astfel încât arborii expresiilor booleene de mai jos să corespundă convențiilor de parsare obișnuite din limbajele de programare:

```
Check "x" < ' "y".
Check "x" = ' "y".
Check ! "x" = ' "y".
Check "x" < ' "y" & ' "x" = ' "y".
Check "x" < ' "y" | ' "x" = ' "y".
Check "x" > ' "y" | ' "x" = ' "y".
Check "x" > ' "y".
Check 2 > ' "y".
Check 3 > ' "y".
Check "x" <= ' "y".
Check 3 <= ' "y".
Check 2 <= ' "y".
Check "x" >= ' "y".
Check 2 >= ' "y".
Check 1 >= ' "y".
Check !( ! "x" < ' "y" & ' ! "x" = ' "y").
Check ("x" < ' "y" | ' "x" = ' "y") .
```

Atenție! Nu modificați tipul de date `Cond` adăugând constructori noi pentru operatorii noi (care apar în exemplele de mai sus și care nu au un constructor corespunzător). În schimb, puteți adăuga notații pentru operatorii care nu au un corespondent direct printre constructorii lui `Cond`.

3. (0.5 puncte) Completați tipul de date de mai jos pentru instrucțiunile unui limbaj de programare simplu. Instrucțiunea `skip` este instrucțiunea care corespunde blocului vid de instrucțiuni, `assign` corespunde atribuirilor, `seq` modelează blocuri de instrucțiuni, `ite` este instrucțiunea decizională cu două ramuri de tip *if-then-else*, iar `while` este utilizat pentru bucle de program. Pentru completarea spațiilor goale de mai jos ... puteți utiliza tipurile `AExp` și `Cond` definite la exercițiile anterioare.

```
Inductive Stmt :=
| skip : Stmt
| assign : string -> AExp -> ...
| seq : Stmt -> Stmt -> Stmt
| ite : ...
| while : ... .
```

4. (0.5 puncte) Utilizați Coercion și Notation astfel încât programele de mai jos să poată fi parsate ca `Stmt`:

```
Definition compute_modulo (x y : string) :=
"m" ::= 0 ;;
while (x > ' y) (
  x ::= x - ' y
);;
ite (x = ' y) ("m" ::= 0) ("m" ::= x) ;;
"result" ::= "m".
```

și

```
Definition is_even (x : string) :=  
  ite (x / ' 2 = ' 0) ("is_even" ::= 1) ("is_even" ::= 0).
```

5. (1 punct) Scrieți un program (ce poate fi parsat ca `stmt`) care calculează cel mai mare divizor comun a două numere utilizând algoritmul lui Euclid prin scăderi repetate.
6. (1 punct) Scrieți un program (ce poate fi parsat ca `stmt`) care calculează cel de-al n -lea termen al șirului Fibonacci.

Coq Reference Manual: <https://coq.inria.fr/distrib/current/refman/>

Last updated: 01/09/2025 09:05:10 - Copyright © Andrei Arusoaie

Facultatea de Informatică, Universitatea „Alexandru Ioan Cuza” din Iași